

Software Requirements Specification

Collaborative Study Material & Quiz Platform

Version 1.0

Group 5

Course: Software Engineering (UE24CS341A)

Faculty: Dr. Sudeepa Roy Dey, Associate Professor

Department of Computer Science and Engineering, PES University

Prepared by:

Bharat Shandilya (PES2UG24CS906)

Sandeep H (PES2UG24CS440)

Samyak Sanklecha (PES2UG24CS436)

Shreyas B Patil (PES2UG24CS488)

04 September 2026

Revision History

Name	Date	Reason For Changes	Version
Group 5	04 September 2026	Initial version of the SRS	1.0

1. Introduction

1.1 Purpose

This document describes the software requirements for the Collaborative Study Material & Quiz Platform, Version 1.0, which we are building as our Software Engineering mini-project. It lays out what the platform should do, who will use it and how it will run, so that all four of us share the same understanding while implementing it.

1.2 Intended Audience and Reading Suggestions

This document is written for our course faculty and evaluators, and for us as the development team. Section 2 gives a quick overview; Section 5 covers the functional requirements and Section 6 the non-functional ones.

1.3 Product Scope

We are building a web platform where students can share study material, build a topic-wise question bank, create and attempt quizzes, and track their performance - one place to prepare together and self-assess instead of scattering notes and questions across apps. For this version we keep the scope to a working system that runs and is demonstrated on localhost; we are not deploying it publicly.

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- MongoDB, Express.js, React and Node.js official documentation (MERN stack).
- Course project guidelines shared by the faculty, Software Engineering (UE24CS341A).

2. Overall Description

2.1 Product Perspective

This is a new, self-contained web application built from scratch. It is not a replacement for our college LMS and does not connect to any existing university system. It has a React front end, a Node/Express back end and a MongoDB database.

2.2 Product Functions

At a high level, the platform will let users:

- Upload, share and browse study materials organised by subject and topic.
- Contribute questions to a shared, topic-based question bank.
- Create quizzes from the question bank and attempt them.
- View their scores and topic-wise performance over time.

- Give feedback and ratings on shared materials and questions.

2.3 User Classes and Characteristics

We expect three main user classes:

- Student – the primary user, who browses materials, attempts quizzes and tracks performance. Most of our users fall in this class.
- Contributor – a student who additionally uploads materials, adds questions and creates quizzes. This is the same login as a student, just doing more.
- Admin – manages users, content and subjects/topics, and keeps the platform clean.

2.4 Operating Environment

The application is built on the MERN stack (MongoDB, Express.js, React, Node.js). For this version we run and demonstrate it on localhost from our own laptops, accessed through a modern browser such as Chrome. No special hardware is needed beyond a normal laptop.

2.5 Design and Implementation Constraints

- We will implement the system using the MERN stack, with JavaScript on both front and back end and MongoDB as the database.
- The system will run locally for the current version; we are not setting up cloud hosting or a public domain.
- We will use GitHub for version control and Jira for task tracking during development.
- As a student team working within a semester, we are keeping the feature set focused and realistic.

2.6 Assumptions and Dependencies

- We assume users have a working internet browser and a stable local setup.
- We assume the demonstration is done on localhost during the review.
- The project depends on open-source MERN components remaining freely available.

3. External Interface Requirements

3.1 User Interfaces

The main screens we plan to build are: login/registration, a materials page (browse/search/upload), a question bank page, a quiz page (create and attempt) and a performance dashboard. We keep navigation simple with a common top bar and clear buttons, and show readable error messages when an action fails.

3.2 Software Interfaces

The React front end talks to the Node/Express back end through REST API calls, and the back end reads and writes to MongoDB. Uploaded materials (PDFs, presentations, etc.) are stored and retrieved through the back end.

3.3 Communications Interfaces

The client and server communicate over HTTP using JSON. As we run on localhost, this is between the browser and the local server.

4. Analysis Models

The use case diagram below shows the main actors (Student, Contributor and Admin) and how they interact with the platform.

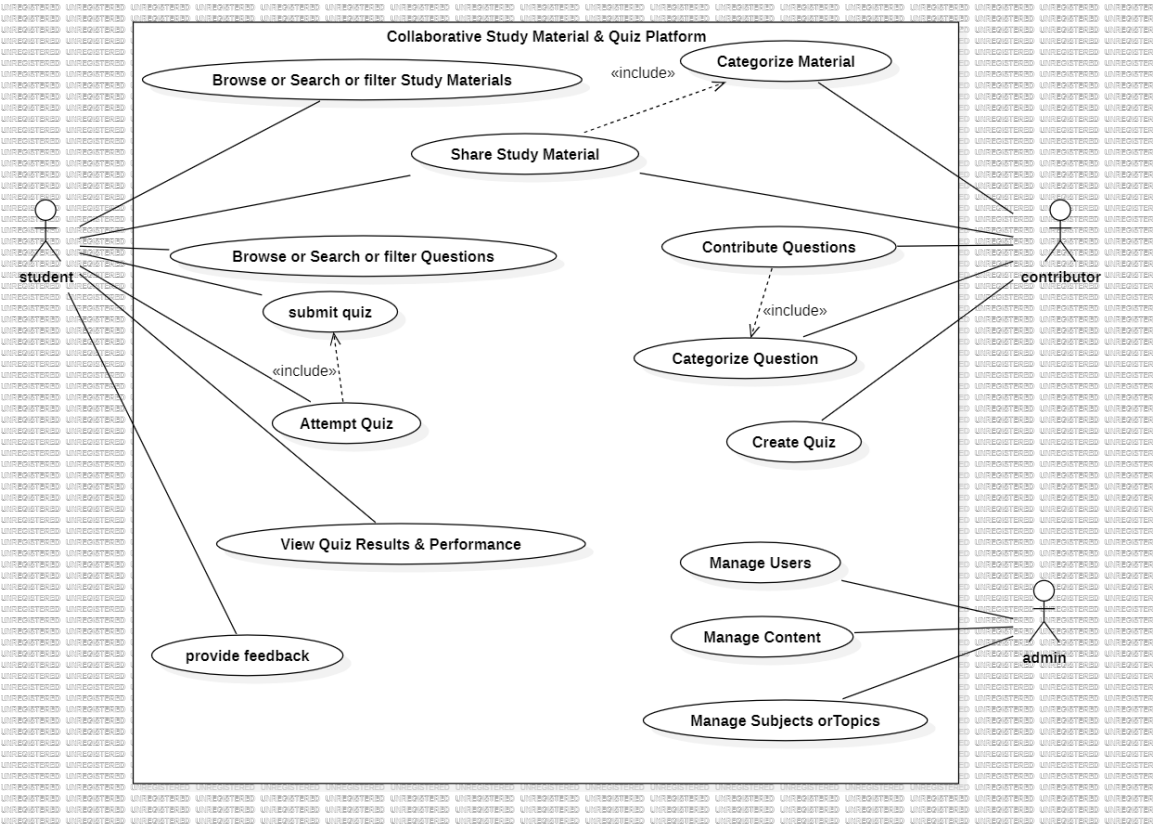


Figure 1: Use Case Diagram

5. System Features

5.1 Collaborative Study Material Sharing

Description and Priority: Students can upload and share study materials such as notes, PDFs and presentations, and browse or search materials shared by others. Priority: High.

Stimulus/Response: A user selects a file, adds a subject/topic and uploads it; the system stores it and lists it for others to view. Another user searches by subject or topic and the system returns the matching materials.

Functional Requirements:

REQ-1: The system shall let a logged-in user upload study material and tag it with a subject and topic.

REQ-2: The system shall let users browse, search and filter materials by subject and topic.

REQ-3: The system shall show a clear error message if an upload fails or an unsupported file type is selected.

5.2 Topic-Based Question Bank

Description and Priority: Students can contribute questions to a shared question bank organised by subject, topic, difficulty and type, and can search and filter them. Priority: High.

Stimulus/Response: A user adds a question with its topic and difficulty; the system saves it to the bank. Another user filters by topic and the system lists the matching questions.

Functional Requirements:

REQ-4: The system shall let a logged-in user add a question with subject, topic, difficulty level and type.

REQ-5: The system shall let users search and filter questions by subject, topic and difficulty.

REQ-6: The system shall store all questions centrally so they are available for quiz creation.

5.3 Quiz Creation and Attempting

Description and Priority: Users can create quizzes from the question bank and attempt them within a set time, and the system evaluates objective questions automatically. Priority: High.

Stimulus/Response: A user picks a topic and creates a quiz; the system selects questions from the bank. A student attempts the quiz and submits it; the system evaluates and shows the result.

Functional Requirements:

REQ-7: The system shall let a user create a quiz by selecting a topic and number of questions from the question bank.

REQ-8: The system shall let a student attempt a quiz and submit answers.

REQ-9: The system shall automatically evaluate objective-type questions and display the score after submission.

5.4 Performance Analysis and Progress Tracking

Description and Priority: The system analyses quiz attempts to show scores, accuracy and topic-wise performance, and lets students track improvement across multiple quizzes. Priority: Medium.

Stimulus/Response: After a student submits quizzes, the system records the results and shows a performance summary. The student opens the dashboard and the system displays their scores and topic-wise breakdown.

Functional Requirements:

REQ-10: The system shall record each quiz attempt with its score and topic.

REQ-11: The system shall display a student's average score and topic-wise performance.

REQ-12: The system shall let a student view their performance across multiple quizzes over time.

5.5 Collaborative Content Evaluation and Feedback

Description and Priority: Users can give feedback or ratings on shared materials and questions, which helps highlight useful and good-quality resources. Priority: Medium.

Stimulus/Response: A user rates or comments on a material or question; the system saves the feedback and updates the resource's rating for others to see.

Functional Requirements:

REQ-13: The system shall let a logged-in user rate and give feedback on materials and questions.

REQ-14: The system shall display the average rating of a material or question to help users find good resources.

5.6 Administration

Description and Priority: An admin manages users, moderates shared content and maintains the list of subjects/topics, keeping the platform clean and organised. This is a shared platform-level feature that we maintain together rather than a single member's module. Priority: Medium.

Stimulus/Response: An admin opens the admin section and can view or remove a user, take down inappropriate material or questions, and add or edit the available subjects/topics.

Functional Requirements:

REQ-15: The system shall let an admin view and manage user accounts.

REQ-16: The system shall let an admin moderate (review and remove) shared materials and questions.

REQ-17: The system shall let an admin add, edit or remove subjects and topics used to organise content.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

Since the platform runs on our local setup for this version, we are keeping our targets realistic:

- Common actions such as searching materials, loading questions and opening a quiz should complete within about 2-3 seconds under normal use.
- When a student submits a quiz, the score should be calculated and shown within about 2 seconds.
- The performance dashboard should load a student's results within about 3 seconds.
- We expect the system to comfortably handle around 20-30 users at a time, which matches a typical class using it together.

6.2 Safety Requirements

This is an academic platform and does not control any physical process, so there are no specific safety hazards. As a basic safeguard, we will avoid permanent loss of user data by storing everything in the database rather than only in memory.

6.3 Security Requirements

- Users must register and log in before they can upload material, add questions or attempt quizzes.
- Passwords will be stored in hashed form, not as plain text.
- Admin-only actions such as managing users and content will be restricted to the admin role.

6.4 Software Quality Attributes

- Usability: the interface should be simple enough that a student can find material, attempt a quiz and view results with minimal navigation.
- Reliability: quiz submissions and scores should be saved correctly and not be lost on a page refresh.
- Maintainability: we will keep the code modular (separate front end, back end and database layers) and track it on GitHub so it is easy to extend.
- Scalability: the design should allow more users, materials and questions to be added later without redesigning the core.

6.5 Business Rules

- Only logged-in users can contribute materials, questions, quizzes or feedback.
- Only admins can manage users, content and the list of subjects/topics.
- A student's performance data is shown to that student on their own dashboard.

7. Other Requirements

All application data (users, materials, questions, quizzes, results and feedback) will be stored in MongoDB. We will use GitHub for version control and Jira to track our tasks and sprints during development. No internationalisation or payment features are planned for this version.

Appendix A: Glossary

MERN: MongoDB, Express.js, React, Node.js - the technology stack we are using.

Question Bank: A shared, topic-wise collection of questions used to build quizzes.

Quiz Attempt: A single instance of a student taking and submitting a quiz.

Performance Dashboard: The screen that shows a student's scores and topic-wise results.

LMS: Learning Management System.

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description	Module Reference
1	REQ-1..3	Study material sharing	Materials module
2	REQ-4..6	Topic-based question bank	Question bank module
3	REQ-7..9	Quiz creation and attempting	Quiz module
4	REQ-10..12	Performance analysis	Analytics module
5	REQ-13..14	Feedback and ratings	Feedback module
6	REQ-15..17	Administration (users, content, topics)	Admin module